Projet Hôtel modulaire

Application console de gestion d'un hôtel

MATSINKOU TAKOUDJOU MEGANE KASSANDRA

LocalHost Academy
Programme Python Full-Stack

Semaine 11 – Jour 3

Plan de la présentation

1. Présentation du projet
2. Données et organisation
3. Fonctions de gestion
4. Menu et vérification
5. Conclusion

Contexte et objectif

Contexte

Un petit hôtel souhaite enregistrer ses clients, consulter ses chambres et gérer les réservations.

Objectif : construire une application console en Python, organisée en modules, qui permet de consulter les chambres et de gérer les réservations.

Le mini-projet est composé de trois exercices qui forment les étapes d'une même application :

1. Importer la base et organiser le projet.
2. Développer les fonctions de gestion.
3. Construire le menu interactif.

Périmètre fonctionnel

Consultation

- Lister les chambres.
- Lister les réservations.
- Afficher les informations du séjour.

Gestion des réservations

- Ajouter une réservation.
- Modifier ses dates.
- Supprimer après confirmation.

Limite annoncée dans le sujet

Le projet ne vérifie pas encore les chevauchements de réservations pour une même chambre.

Modèle de données

La base SQLite repose sur trois tables liées :

Table	Attributs
`clients`	`id`, `nom`, `email`
`chambres`	`id`, `numero`, `prix_nuit`
`reservations`	`id`, `client_id`, `chambre_id`, `date_debut`, `date_fin`

Une réservation concerne un seul client et une seule chambre. Un client et une chambre peuvent apparaître dans plusieurs réservations à des dates différentes.

Importation et préparation de la base

1. Ouvrir DB Browser for SQLite.
2. Choisir **Fichier** → **Importer** → **Base de données depuis un fichier SQL**.
3. Sélectionner le fichier fourni `hotel.sql`.
4. Créer une nouvelle base et l'enregistrer sous `hotel.db`.

Précaution

L'importation se fait dans une base neuve : le script crée déjà les tables et contient les données initiales. Il ne faut pas le réexécuter sur une base qui contient déjà ces tables.

Organisation des fichiers

Structure du projet

```
projet-hotel/
|-- hotel.sql
|-- db.py
|-- models.py
|-- main.py
|-- verification_models.py
|-- recette.md
|-- README.md
`-- .gitignore
```

- `hotel.sql` : script fourni pour reconstruire la base.
- `db.py` : chemin de la base et ouverture de connexion.
- `models.py` : fonctions d'accès aux données et opérations métier.
- `verification_models.py` : tests manuels des fonctions.
- `main.py` : menu et dialogue avec l'utilisateur.
- `recette.md` : scénarios et résultats de recette.

Connexion à la base de données

Le module `db.py` utilise `Path` pour construire un chemin indépendant du dossier depuis lequel la commande est lancée.

```
from pathlib import Path
import sqlite3

DB_PATH = Path(__file__).resolve().parent / "hotel.db"
```

- `__file__` désigne le fichier Python courant.
- `resolve()` fournit son chemin absolu.
- `parent` désigne le dossier qui contient ce fichier.
- `get_connection()` ouvre la connexion, active `PRAGMA foreign_keys = ON`, puis la retourne.

L'appelant est responsable de fermer la connexion.

Fonctions de consultation

Le module `models.py` fournit des fonctions réutilisables sans saisie utilisateur ni affichage.

Lister les chambres

Retourne des tuples `(id, numero, prix_nuit)`, triés par numéro. Si aucune chambre n'existe, le résultat est une liste vide.

Lister les réservations

Utilise deux jointures pour retourner `(id, nom_client, numero_chambre, date_debut, date_fin)`, triés par identifiant de réservation.

CRUD des réservations

Créer	Enregistre une réservation et retourne son identifiant.
Modifier	Modifie les dates d'une réservation et retourne le nombre de lignes modifiées : 0 ou 1.
Supprimer	Supprime une réservation et retourne le nombre de lignes supprimées : 0 ou 1.

Règles de validation

- Requêtes paramétrées avec des marqueurs ?.
- Dates au format `AAAA-MM-JJ`, validées avec `date.fromisoformat()`.
- La date de fin doit être strictement postérieure à la date de début.
- Les clés étrangères doivent être respectées.
- Les écritures sont validées au succès et annulées en cas d'erreur.

Menu de l'application

Gestion de l'hôtel

1. Lister les chambres
2. Lister les réservations
3. Ajouter une réservation
4. Modifier les dates d'une réservation
5. Supprimer une réservation

0. Quitter

- `main.py` gère les échanges avec l'utilisateur.
- La logique SQL reste dans `models.py`.
- La connexion est ouverte au début de `main()` et fermée dans un bloc `finally`.

Gestion des erreurs et tests

Erreurs à prendre en charge :

- Identifiant non numérique ou inférieur ou égal à zéro.
- Date invalide ou période incohérente.
- Violation d'une clé étrangère ou d'une autre contrainte SQL.
- Identifiant de réservation inexistant.
- Choix de menu inconnu.

Vérifications prévues :

- Consulter les trois chambres et les deux réservations initiales.
- Créer, modifier puis supprimer une réservation de test.
- Vérifier le retour 0 pour la réservation 999.
- Vérifier les erreurs de saisie et la persistance après relance.
- Lancer le programme depuis le dossier parent.

Pourquoi protéger le lancement de main.py ?

```
if __name__ == "__main__":
    main()
```

Cette garde lance le menu lorsque `main.py` est exécuté directement. Lorsqu'il est importé par un autre module, le menu ne démarre pas automatiquement.

Cela permet de réutiliser le contenu du module sans déclencher l'interaction avec l'utilisateur.

Livraison du projet

Le dépôt GitHub doit contenir :

- `hotel.sql`, `db.py`, `models.py`, `main.py`
- `verification_models.py`, `recette.md`, `README.md`
- `.gitignore`

Base locale

`hotel.db` ne doit pas être publiée. Elle est reconstruite à partir du fichier `hotel.sql`. Le fichier `.gitignore` doit notamment exclure les fichiers `*.db`, les caches Python et les environnements virtuels.

Merci pour votre attention

Projet Hôtel modulaire – Python / SQLite